21CSE525T – AGENT TECHNOLOGY

ASSIGNMENT 5.2

Capstone Project Consolidation and Secure Deployment

CipherMesh: The Privacy Control Plane for Enterprise AI

Submitted by

KUMAR WAIBHAV AKSHAT [RA2212704010027]

M.Tech (Integrated) Computer Science & Engineering
with specialization in Data Science

Under the Guidance of

Dr. Prabhu Kavin B

Assistant Professor
Department of Data Science and Business Systems

DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603 203

NOVEMBER 2026

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Objective 1
1.2 The platform journey from Unit 1 to Unit 5 1
1.3 The problem in CipherMesh terms 1
2 REQUIREMENTS AND TRACEABILITY 2
3 UNIFIED ARCHITECTURE 3
3.1 System schematic 3
3.2 Unit to module map 3
3.3 AgentHost internals 4
3.4 Host topology and trust model 5
3.5 Coordination protocol 6
4 IMPLEMENTATION 6
4.1 Modules 6
4.2 Key code 7
4.3 UI layer 8
4.4 Deployment 10
5 PRODUCTION EXECUTION CYCLE 11
5.1 How the cycle runs 11
5.2 P0 Boot and self-attestation 12
5.3 P1 Live knowledge 12
5.4 P2 Autonomous processing 13
5.5 P3 Coordination and load sharing 14
5.6 P4 Environmental change: provider outage 14
5.7 P5 Security 15
5.8 P6 Lockdown drill and graceful shutdown 16
5.9 Results across the cycle 16
6 TESTING 17
7 VIDEO PRESENTATION 17
8 SUBMISSION ANALYSIS 18
8.1 Requirement, evidence and status 18
8.2 Strengths 19
8.3 Honest limitations 19
8.4 Lessons learned 19
8.5 Future work 19
9 CONCLUSION 20
REFERENCES 21
App. A HOW TO RUN 22
App. B FILE LISTING 23
App. C REPOSITORY AND PUBLISHING 24

LIST OF FIGURES

Figure 3.1 CipherMesh v3.0 system architecture: hosts, unit layers, flows and trust boundary.....3
Figure 3.2 Path of one prompt through the layers of an AgentHost.....4
Figure 3.3 Hosts, roles and permitted commands in the capstone trust store.....5
Figure 3.4 Heartbeat, policy sync and Contract-Net load sharing over signed commands.....6
Figure 4.1 Web ops dashboard during P2: pipeline counters, current intention with step progress, beliefs.....8
Figure 4.2 Dashboard during P4: intention shift, circuit breaker and belief sync to the peer.....9
Figure 4.3 Dashboard after the attack battery in P5 (dark theme).....9
Figure 4.4 Final dashboard state after P6 (light theme).....10
Figure 5.1 The seven phases of the production execution cycle.....11
Figure 5.2 Timeline of the recorded run (one tick per event; xN marks a burst).....12
Figure 5.3 End-to-end time per P2 request by decision.....13
Figure 5.4 Hostile frames blocked per checkpoint in P5.....16
Figure 5.5 Policy decisions per phase in the recorded run.....16
Figure 7.1 Key frames: title card, P4 outage on the live dashboard, security stream close-up.....18

LIST OF TABLES

Table 1.1 CipherMesh units and their role in the capstone....1
Table 2.1 Assignment lines, where they are satisfied and the evidence....2
Table 3.1 Unit to module map of the unified codebase....4
Table 3.2 State model of one AgentHost....5
Table 4.1 New modules in backend/ciphermesh/capstone/....6
Table 4.2 Deployment changes for v3.0....11
Table 5.1 Contract-Net capacity contracts negotiated at boot (USD per 1,000 requests)....12
Table 5.2 Live feeds fetched in P1....13
Table 5.3 P2 outcome by decision....13
Table 5.4 Tasks delegated to gateway-beta in P3....14
Table 5.5 Intention shifts recorded in P4....15
Table 5.6 Attack battery against gateway-alpha....15
Table 5.7 Final state of the gateways after the drain....16
Table 6.1 Test counts by unit....17
Table 6.2 Capstone tests....17
Table 7.1 Video storyboard....18
Table 8.1 Requirement, evidence and status....18
Table 9.1 Course concepts and where the capstone demonstrates them....20
Table A.1 Commands that produced each output....22
Table B.1 Main files of the submission....23

1 INTRODUCTION

1.1 Objective

CipherMesh is a provider-neutral privacy agent that sits between enterprise applications and AI providers. It inspects every prompt, blocks secrets, masks personal data, holds internal links for a human and routes the rest to the best provider that the policy allows. Over five units I built this agent one capability at a time. Each unit was tested on its own, in its own folder. This capstone, **CipherMesh v3.0**, joins all of them into one codebase and runs them together as a small, multi-host, production-style system.

The assignment asks for four things: a unified codebase that assembles the core features of Units 1 to 5, a clean UI layer that shows the live state of the agent, a final production execution cycle in which the agent works on its own, coordinates with other hosts, updates its intentions and acts safely, and a set of deliverables (source package, repository link, a 3-minute video and an architecture schematic). This report explains how each one was built and shows the evidence from a real run.

1.2 The platform journey from Unit 1 to Unit 5

An agent perceives its environment and acts on it to reach its goals [1]. Table 1.1 lists what each unit added to CipherMesh and how that piece is used inside one capstone host. The capstone does not rewrite any of them: it imports their public APIs and wires them together.

Table 1.1: CipherMesh units and their role in the capstone

Unit	What it added	Role inside an AgentHost
1.1 / 1.2 Core loop	PEAS model; simple-reflex loop: perceive, match R1-R5, act	Admission reflex on every prompt; policy decides ALLOW, REDACT, REVIEW or BLOCK
2.1 Workers	Bounded queue, non-daemon worker pool, daemon monitor	Runs every request; monitor probes providers and feeds the circuit breaker
2.2 Sockets	Length-prefixed TCP frames, typed envelopes, HMAC	Frame format of every host-to-host and console message
3.1 Cooperation	Contract-Net negotiation of provider capacity	Capacity contracts per provider; load sharing between gateways
3.2 Optimisation	Constrained utility router with residency rules	Chooses the provider inside each BDI plan
4.1 BDI	Beliefs, desires D1-D5, plans, intentions, reconsideration	One deliberation per request; re-plans when a belief changes
4.2 Knowledge	Ontology, triple store, six live feeds, adapter	Turns live facts into beliefs: posture, budget, provider status
5.1 Security	Ed25519 identities, 13 checkpoints, self-attestation	Trust gate on every inbound command; signed replies; start-up attestation

1.3 The problem in CipherMesh terms

Separately, each unit worked. Together, they raise new questions. A belief learned from a live feed must reach a plan that is already running on a worker thread. A host that is too busy should share work, but a peer must never receive a secret or a review-only prompt. Hosts must trust each other enough to share beliefs, yet a stolen peer key must not be enough to lock the whole mesh down. And an operator needs to *see* all of this while it happens. The capstone answers

these questions with one class, `AgentHost`, a small coordination protocol on top of the Unit 5.1 signed commands, a thread-safe `StateBus`, and a web and terminal dashboard.

> **IN PLAIN WORDS**
> Five working parts become one working machine. Two CipherMesh gateways, a security console and an attacker run on real network addresses. The gateways process 82 real requests, share work, learn from live data, change their plans when a provider fails, accept only signed commands and block every attack, while a live dashboard shows each step.

2 REQUIREMENTS AND TRACEABILITY

Table 2.1 copies each line of the assignment brief and shows where it is met and which file proves it. Section 8 returns to the same lines with a status for each.

Table 2.1: Assignment lines, where they are satisfied and the evidence

Brief line (verbatim)	Where it is satisfied	Evidence
Objective: Consolidate multiple modular agent capabilities developed throughout the units into a unified, production-ready system.	`backend/ciphermesh/capstone/` wires Units 1-5; one FastAPI app, Docker, Vercel and CI	`logs/production_cycle.log`, Figure 3.1
Assemble core features (Unit 1 Core Loop, Unit 2 Workers/Sockets, Unit 3 Cooperation, Unit 4 Knowledge/BDI, Unit 5 Security) into a unified codebase.	`AgentHost` in `capstone/host.py`; unit packages in `ciphermesh/{bdi,knowledge,trust}`	`tests/capstone/test_host.py::test_every_unit_is_wired_into_one_host`
Develop a clean UI layer (CLI dashboard or simple web GUI) to visualize the live states of the agent.	Both: web dashboard at `/ops` (SSE) and `python -m ciphermesh.capstone.dashboard`	`logs/dashboard_snapshots/*.png`, `logs/cli_dashboard_snapshot.txt`
Perform a final production execution cycle where the agent independently processes inputs, coordinates with other hosts, updates its intents, and executes actions safely.	`demo/run_production_cycle.py` runs P0-P6 with no manual step	`logs/production_summary.json`, `production_events.jsonl`
Deliverable: Final working source code package	Whole repository folder (backend, frontend, demo, docs)	`logs/pytest_results.txt` (563 passed)
Deliverable: repository link	README and `docs/SUBMISSION.md` (publish steps)	https://github.com/kumarwaibhav/CipherMesh
Deliverable: a 3-minute video presentation outlining the complete lifecycle	`video/CipherMesh_Capstone_Lifecycle.mp4` (3:05)	`video/narration_script.md`, `video/captions.srt`
Deliverable: a concise system architecture schematic	`docs/architecture/CipherMesh_v3_Architecture.png` / `.pdf` / `.html`	Figure 3.1

3 UNIFIED ARCHITECTURE

3.1 System schematic

Figure 3.1 is the architecture schematic delivered with the submission. The red dashed line is the organisation trust boundary. Inside it run the two gateways and the local Ollama tier. Outside it are the security console, the attacker, the enterprise applications, the live knowledge feeds and the external AI providers. Solid blue arrows carry prompts and data; dashed purple arrows carry signed commands; the red cross marks where hostile frames stop. No arrow crosses another.

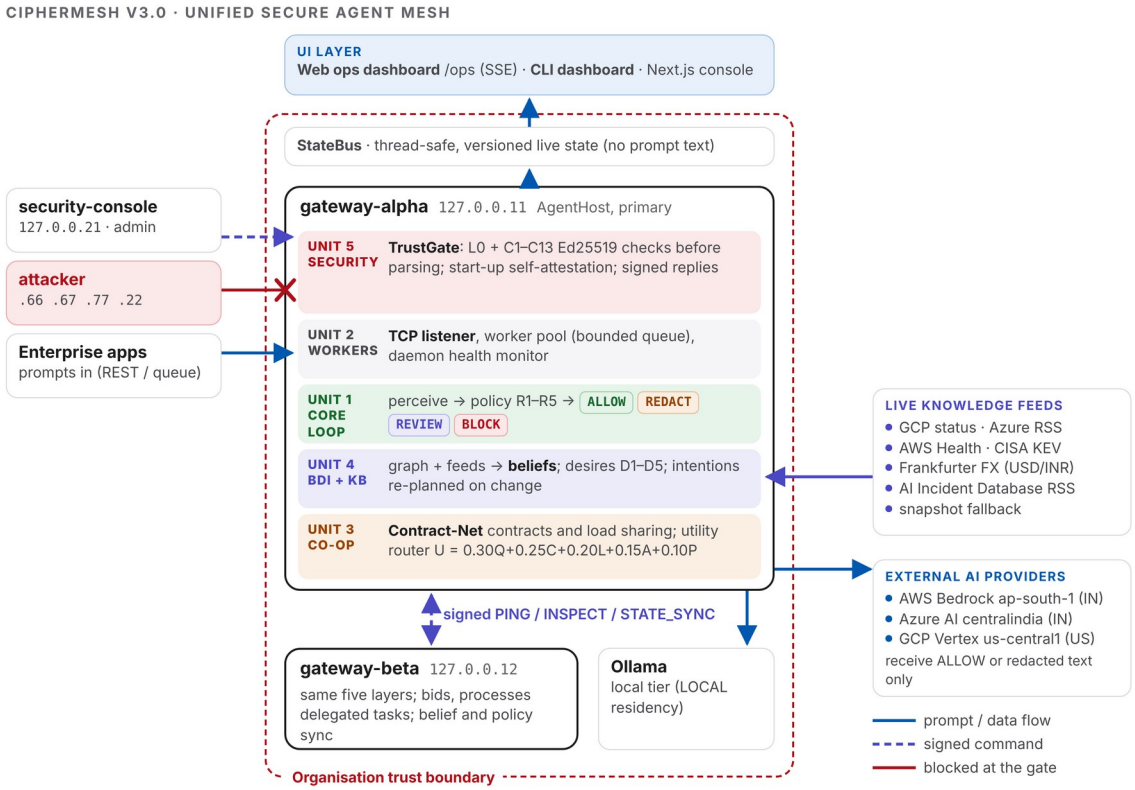


Figure 3.1: CipherMesh v3.0 system architecture: hosts, unit layers, flows and trust boundary

The UI layer never talks to an agent directly. Every host writes its state model to the `StateBus`, and both dashboards read only the bus through `GET /api/v1/ops/state` and the Server-Sent Events stream `GET /api/v1/ops/stream` [2]. A slow or broken browser can therefore never slow down or break a host.

3.2 Unit to module map

Table 3.1 maps each unit to the modules that implement it in the unified codebase. Units 1 to 3 were already in the engine package; Units 4.1, 4.2 and 5.1 are copied unchanged from their final submissions; everything in `capstone/` is new.

Table 3.1: Unit to module map of the unified codebase

Unit	Modules (backend/ciphermesh/...)	Used by
1 Core loop	`detector.py`, `policy.py`, `stream.py`	`AgentHost.reflex`, BDI step `prepare_payload`
2 Workers / sockets	`runtime.py`, `transport.py`, `server.py`, `client.py`, `security.py`	worker pool, health monitor, frame format
3 Cooperation	`negotiation/`, `procurement.py`, `optimization/`, `routing.py`	contracts, router, `coordination.py`
4 Knowledge / BDI	`bdi/` (beliefs, desires, plans, intentions, engine), `knowledge/` (ontology, graph, feeds, adapter)	`DeliberationProcessor`, `BeliefSink`, `KnowledgeBridge`
5 Security	`trust/` (keys, envelope, verifier, integrity, secure agent)	`TrustGate`, `provision_mesh`, attack battery
Capstone (new)	`capstone/`: host, deliberation, beliefs, coordination, trustmesh, statebus, mesh, actors, cycle, dashboard, workload, providers	demo, FastAPI ops routes, tests

3.3 AgentHost internals

Figure 3.2 follows one prompt through a host. The admission reflex is the pure Unit 1 policy call, so the decision is known before any work is queued. If the queue has reached its high-water mark and the prompt may leave the host, it is offered to a peer; otherwise it goes to the Unit 2 worker pool. Each worker hands the prompt to a fresh Unit 4.1 `BDIAgent`. The agent copies the host's world beliefs, keeps receiving their changes while it runs, chooses an intention and executes its plan one step per cycle. The plan steps call the real Unit 3 procurement and router and the simulated provider adapters, and inspect the reply before it is delivered.

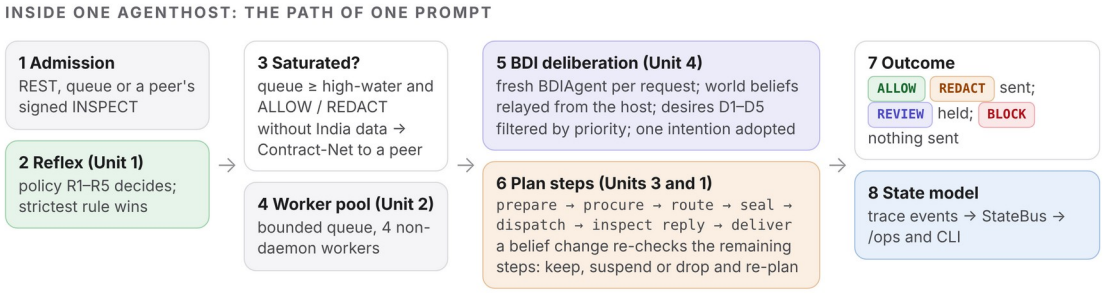


Figure 3.2: Path of one prompt through the layers of an AgentHost

Table 3.2 lists the pieces of state that one host keeps and publishes. This is the *state model* the assignment asks the UI to visualise.

Table 3.2: State model of one AgentHost

Part	Source	Shown on the dashboard as
Beliefs snapshot	host `BeliefBase` (engine seed, feeds, monitor, peers)	beliefs table; recent changes highlighted
Desires D1-D5	`ciphermesh_desires()` with live activity	desire list with priority, kind, active state
Intentions	live trace events of every running `BDIAgent`	current intention with step cursor; shift timeline
Queue and workers	`ConcurrentRuntime` metrics	host cards (queue, workers, completed)
Provider health	circuit breaker + belief `provider.<id>.status`	provider table with utility bars
Contracts and load sharing	`ContractBook`, task records	Contract-Net counters and task list
Threat posture and feeds	knowledge adapter records	posture badge, FX, budget, R5 limit, feed table
Security events	trust gate verdicts (no payloads)	ACCEPTED / BLOCKED stream with checkpoint
Peers and counters	signed heartbeats, decision counters	host cards, pipeline tiles, event log

3.4 Host topology and trust model

The four hosts run on real TCP loopback with distinct 127.0.0.x addresses. Each client binds its source address before it connects, so checkpoint C7 compares the peer address that the kernel reports with the CIDR blocks in the trust store. Figure 3.3 shows each host's role and which commands that role may send. The roles come from Unit 5.1 and follow least privilege [3]: a gateway can ping, ask for an inspection and synchronise four non-weakening policy fields, but it can never lock down or change redaction patterns. Only the security console holds those rights.

HOSTS, ROLES AND WHAT EACH ONE MAY SEND (CHECKED AT C13)

Host	Address	Role	PING	INSPECT	STATE_SYNC	Admin commands
gateway-alpha	127.0.0.11/32	peer-gateway	yes	yes	4 fields only	no
gateway-beta	127.0.0.12/32	peer-gateway	yes	yes	4 fields only	no
security-console	127.0.0.20/30	security-admin	yes	no	all fields	lockdown, release, patterns, telemetry
attacker	.66 .67 .77 (+ .22, + stolen key)	not in trust store	no	no	no	no

The four peer fields are policy_revision, policy_version, provider_allowlist and max_prompt_bytes. A gateway applies a peer's prompt limit only if it is stricter. Every reply is signed by the replying host with the REPLY domain context.

Figure 3.3: Hosts, roles and permitted commands in the capstone trust store

Every key is an Ed25519 key [4] generated at start-up into a runtime directory with mode 0600, encrypted with a passphrase and never shipped. The release key signs a manifest of every source file; each gateway attests its own code against it before it starts, in the style of the RATS architecture [5]. A host that finds a changed file, a missing file or any compiled artefact next to its sources starts in SAFE MODE.

3.5 Coordination protocol

Unit 5.1 defines a fixed command vocabulary, and its final audit made `STATE_SYNC` field-level: a peer gateway may only send `policy_revision`, `policy_version`, `provider_allowlist` and `max_prompt_bytes`. The capstone therefore adds no new command. Figure 3.4 shows how heartbeats, policy sync and Contract-Net load sharing [6] map onto `PING`, `STATE_SYNC` and `INSPECT`, with all data that a host *reports* carried in its signed reply.

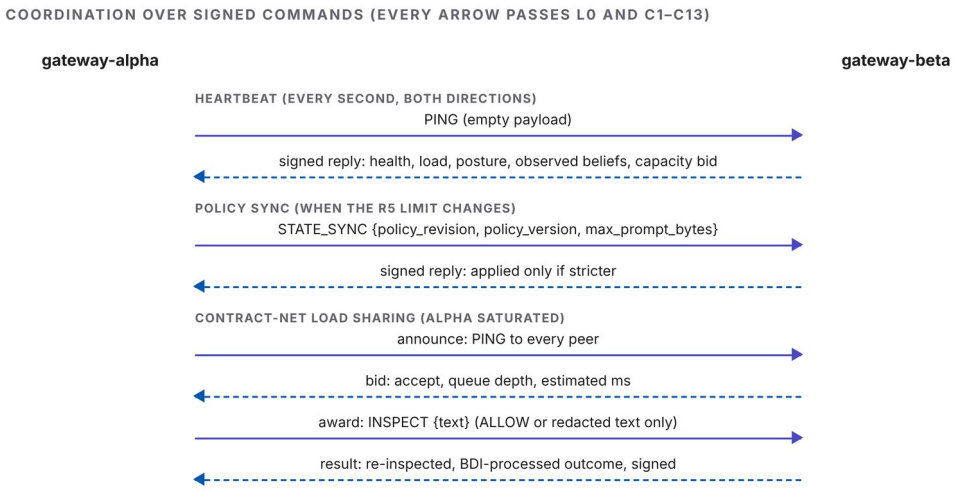


Figure 3.4: Heartbeat, policy sync and Contract-Net load sharing over signed commands

Three rules keep sharing safe. First, only ALLOW text or REDACT-sanitised text may be awarded, and never text whose original findings include Aadhaar, PAN or an Indian phone number, because desire D2 keeps Indian personal data on the home host. Second, the receiving gateway re-inspects the awarded text with its own policy and refuses anything that is not ALLOW or REDACT. Third, a peer's belief snapshot only contains *observed* knowledge (feed results, monitor results, derived posture and budget), never engine seeds, local configuration or beliefs that were themselves learned from a peer, so a belief cannot echo back and overwrite its own source.

4 IMPLEMENTATION

4.1 Modules

Table 4.1 lists the new `capstone` package. It has 12 modules and about 2,300 lines; the unit packages it uses are unchanged.

Table 4.1: New modules in backend/ciphermesh/capstone/

File	Purpose	Key classes / functions
`host.py`	one autonomous node: trust gate, listener, worker pool, BDI, knowledge, coordination, state model	`AgentHost`
`deliberation.py`	per-request BDI run behind the worker pool; live trace forwarding	`DeliberationProcessor`, `HostOutcome`
`beliefs.py`	knowledge -> beliefs bridge; host-to-request belief relay	`BeliefSink`, `KnowledgeBridge`, `BeliefRelay`
`coordination.py`	heartbeat, bid and task-result schemas; sharing rules	`Heartbeat`, `Bid`, `TaskResult`, `may_delegate`

File	Purpose	Key classes / functions
`trustmesh.py`	mesh provisioning, trust gate, signed transport	`provision_mesh`, `TrustGate`, `verify_reply`
`statebus.py`	thread-safe versioned state for the UI	`StateBus`, `default_bus`
`mesh.py`	builds and connects the hosts; publishes state 4 times a second	`Mesh`
`actors.py`	security console and attacker hosts	`SecurityConsole`, `Attacker`
`providers.py`	simulated adapters with a labelled outage switch	`SwitchableAdapter`, `host_registry`
`cycle.py`	production execution cycle P0-P6 and its outputs	`ProductionCycle`
`dashboard.py`	CLI dashboard (standard library only)	`render`, `main`
`workload.py`	synthetic enterprise prompts with expected decisions	`BATCH`, `BURST`, `OUTAGE`

4.2 Key code

Listing 1 is the heart of the BDI integration. Each request gets its own belief base, filled from the host's world beliefs, and a `BeliefRelay` keeps forwarding host changes while the deliberation runs. This is how a provider outage that the health monitor detects on another thread reaches an intention that is already half-way through its plan.

```
def process(self, text, provider=None, request_id=None) -> HostOutcome:
    request_id = request_id or f"req-{uuid4().hex[:10]}"
    beliefs = BeliefBase()
    relay = BeliefRelay(self.world, beliefs)
    agent = BDIAgent(services=self.services, beliefs=beliefs,
                     tracer=LiveTracer(request_id, self.on_trace),
                     max_cycles=self.max_cycles)
    ...
    try:
        context = agent.submit(text, request_id)
        result = agent.run(environment)
    finally:
        relay.close()
    return self._outcome(agent, context, result)
```

Listing 1: DeliberationProcessor.process (capstone/deliberation.py, abridged)

Listing 2 shows the two sharing rules that the coordination code enforces before anything leaves a host, and the policy-sync payload that stays inside the Unit 5.1 field allow-list.

```
def may_delegate(decision: str, categories) -> bool:
    return decision in SHARED_DECISIONS and not (set(categories) & INDIA_CATEGORIES)

def shareable(key: str, value, source: str) -> bool:
    """Observed knowledge only: never engine seeds, local config, or peer beliefs."""
    return (shareable_key(key) and isinstance(value, (str, int, float, bool))
            and not source.startswith(LOCAL_ONLY_SOURCES) and "peer:" not in source)

def policy_sync_payload(revision, version, max_prompt_bytes) -> dict:
    return {"state": {"policy_revision": int(revision), "policy_version": str(version),
                      "max_prompt_bytes": int(max_prompt_bytes)}}
```

Listing 2: sharing rules (capstone/coordination.py)

Listing 3 is the bridge from the Unit 2.1 health monitor to the BDI belief base. When the circuit breaker opens, the monitor writes the belief `major_outage`; when a probe succeeds again, it

restores `operational`, but only for a belief that a monitor wrote, so it never overrides a status page.

```
for pid, health in (record.get("provider_health") or {}).items():
    circuit = health.get("circuit", "CLOSED")
    ...
    if circuit == "OPEN":
        self.event("CIRCUIT", f"circuit breaker OPEN for {pid} ...", "block")
        self.world.set(key, "major_outage", "monitor:circuit-breaker")
    else:
        held = self.world.belief(key)
        if held and held.source.startswith("monitor:") and held.value != "operational":
            self.world.set(key, "operational", "monitor:circuit-breaker")
```

Listing 3: AgentHost._on_monitor_sample (capstone/host.py, abridged)

4.3 UI layer

Web ops dashboard. `GET /ops` returns one self-contained HTML page with inline CSS, JavaScript and SVG logo, so it works offline with no build step and no CDN. The page loads the state once and then follows the SSE stream; the server sends at most four state events per second. It uses the CipherMesh brand colours and type from the existing console, has a light and a dark theme, and keeps every text pairing at WCAG AA contrast. It is served with a nonce-based Content-Security-Policy [7] that allows only its own inline script and style and no `unsafe-inline`. Figure 4.1 shows the dashboard during P2, in the middle of the batch.

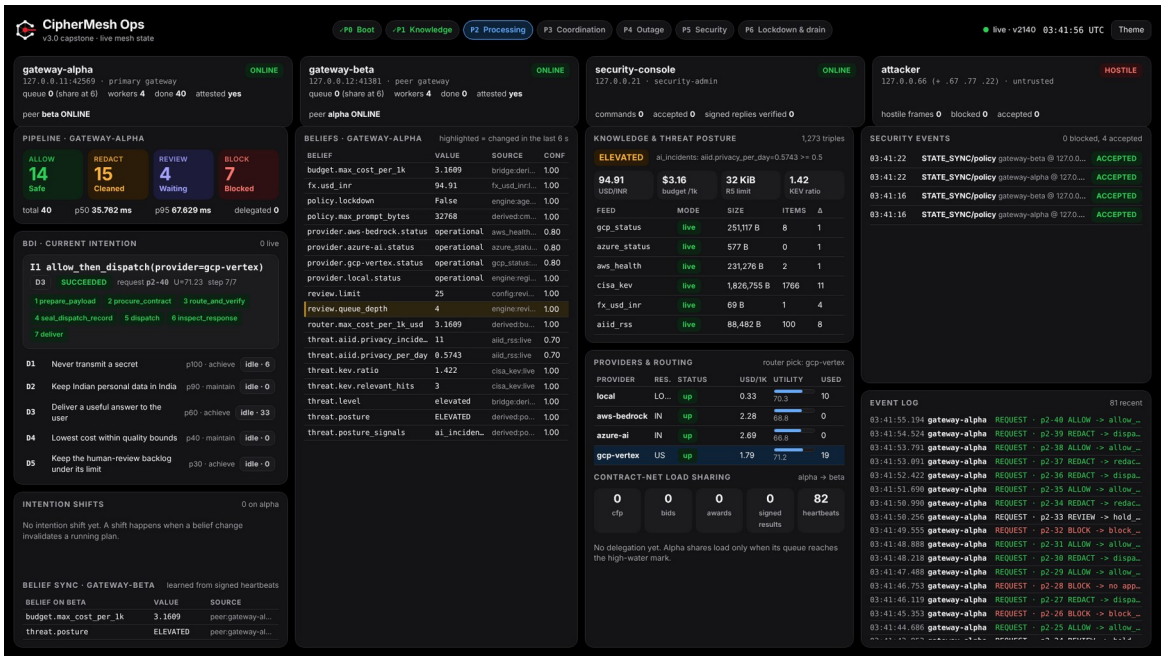


Figure 4.1: Web ops dashboard during P2: pipeline counters, current intention with step progress, beliefs

Figure 4.2 is the same page at the moment of the provider outage in P4. The intention-shift panel shows the dropped plan struck through, the new plan and the belief that caused the change; the beliefs table highlights `provider.gcp-vertex.status`, and the lower table shows that gateway-beta learned the same belief from alpha's signed heartbeat.

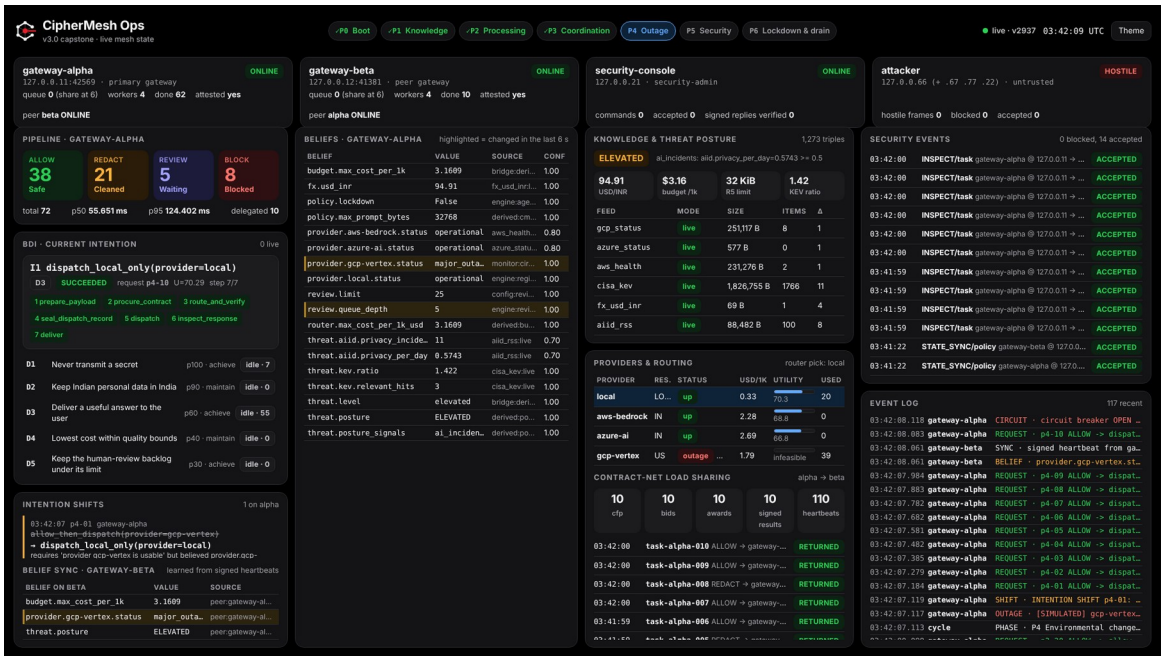


Figure 4.2: Dashboard during P4: intention shift, circuit breaker and belief sync to the peer

Figure 4.3 shows the end of the attack battery in P5. The security stream on the right lists each blocked frame with the checkpoint that stopped it, and the attacker card counts 18 hostile frames, 18 blocked.

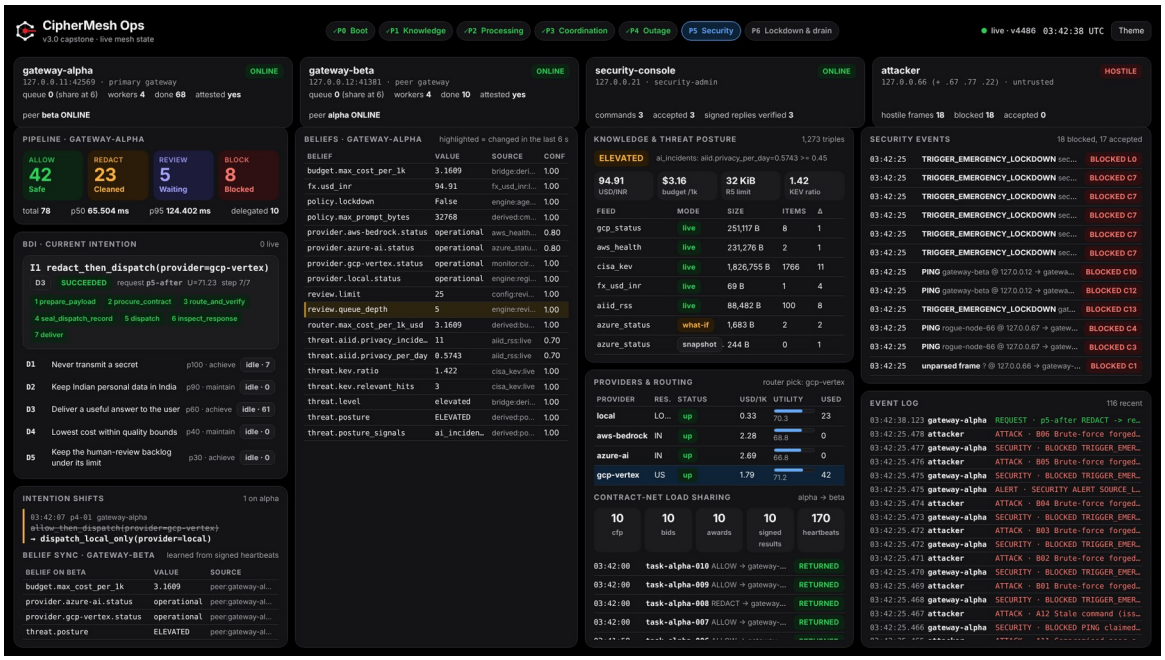


Figure 4.3: Dashboard after the attack battery in P5 (dark theme)

Figure 4.4 is the final state after the graceful drain, in the light theme. Both gateways show STOPPED, the pipeline totals and the Contract-Net counters are final, and the security stream ends with the accepted lockdown and release commands of P6.

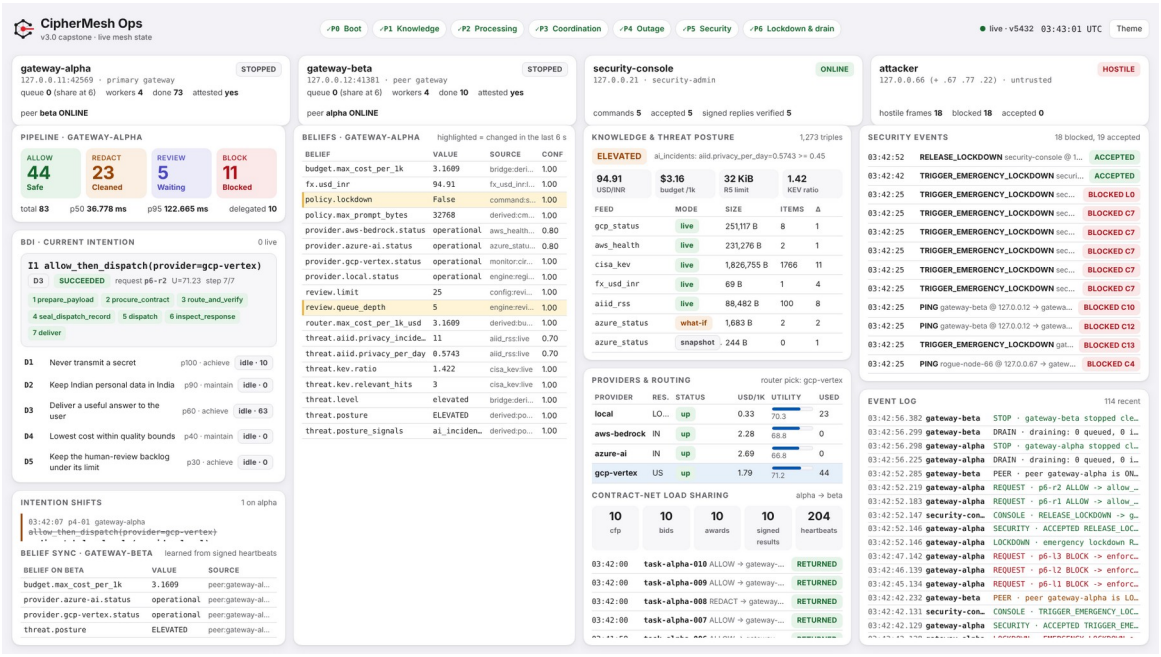


Figure 4.4: Final dashboard state after P6 (light theme)

Protection. When `CIPHERMESH_API_TOKEN` is set, the page, the state endpoint and the stream all require it, either as the `X-CipherMesh-Token` header or as `?token=` (browsers cannot add headers to `EventSource`); the page forwards its own token to its API calls. Without a token the routes are open, like the rest of the development API.

CLI dashboard. `python -m ciphermesh.capstone.dashboard` uses only the standard library. On a terminal it redraws the same state every second with ANSI colour; in a pipe or a file it prints plain snapshots separated by a rule. Listing 4 is the top of the snapshot that the production cycle wrote at the end of the run.

```
CipherMesh v3.0 ops | phase DONE Cycle complete | bus v5432 | 2026-11-03T03:42:56.392Z
------------------------------------------------------------------------------------...
gateway-alpha     STOPPED   queue 0/6 workers 4 done 73 attested True posture ELEVATE...
gateway-beta      STOPPED   queue 0/6 workers 4 done 10 attested True posture ELEVATE...
security-console  ONLINE    commands 5 accepted 5 verified replies 5
attacker          HOSTILE   hostile frames 18 blocked 18 accepted 0
------------------------------------------------------------------------------------...
PIPELINE  ALLOW  44  REDACT  23  REVIEW   5  BLOCK  11   total 83  p50 36.778 ms  p95...
INTENTION I1 allow_then_dispatch(provider=gcp-vertex) [D3] SUCCEEDED [#######] next: ...
DESIRES   D1(p100) D2(p90) D3(p60) D4(p40) D5(p30)   (* = active now)
SHIFT     p4-01: allow_then_dispatch(provider=gcp-vertex) -> dispatch_local_only(prov...
------------------------------------------------------------------------------------...
```

Listing 4: CLI dashboard snapshot (logs/cli_dashboard_snapshot.txt, first lines)

4.4 Deployment

The existing deployment paths still work, and Table 4.2 lists what changed. The FastAPI app keeps every v2 route; `/ops` and `/api/v1/ops/*` are added and read the process-wide bus. On a plain deployment with no cycle attached the dashboard says so instead of showing empty numbers.

Table 4.2: Deployment changes for v3.0

Target	Change	Why
Docker (`backend/Dockerfile`)	copies `data/` too; `PYTHONDONTWRITEBYTECODE=1` already set	feed snapshots for offline fallback; no `.pyc` next to attested sources
Vercel (`backend/vercel.json`)	`includeFiles` adds `data/**`	same reason in the serverless bundle
CI (`.github/workflows/ci.yml`)	pytest with `-B`, then a CLI dashboard smoke run	whole suite incl. units 4-5 and capstone; UI entry point checked
Requirements	`jsonschema==4.26.0` added; `cryptography` already present	ontology schema validation (Unit 4.2)
Frontend (Next.js)	unchanged	existing console keeps working against the same API

5 PRODUCTION EXECUTION CYCLE

5.1 How the cycle runs

One command, `python -B demo/run_production_cycle.py --serve 8000 --pace 2`, runs the seven phases in Figure 5.1 with no manual step. The `--pace` option only adds pauses between steps so a person can follow the dashboard; it changes no decision. The recorded run took 100.96 s, handled 82 requests and wrote `logs/production_cycle.log` (phase banners, timestamps), `logs/production_events.jsonl`, `logs/production_summary.json` and the dashboard screenshots. No log contains prompt text: only the synthetic labels written for the workload, hashes, categories and decisions.



Figure 5.1: The seven phases of the production execution cycle

Figure 5.2 is the timeline of the recorded run, drawn from the event stream. Requests in P2 are spread out by the demo pacing; the P3 burst arrives at once, which is what saturates the queue.

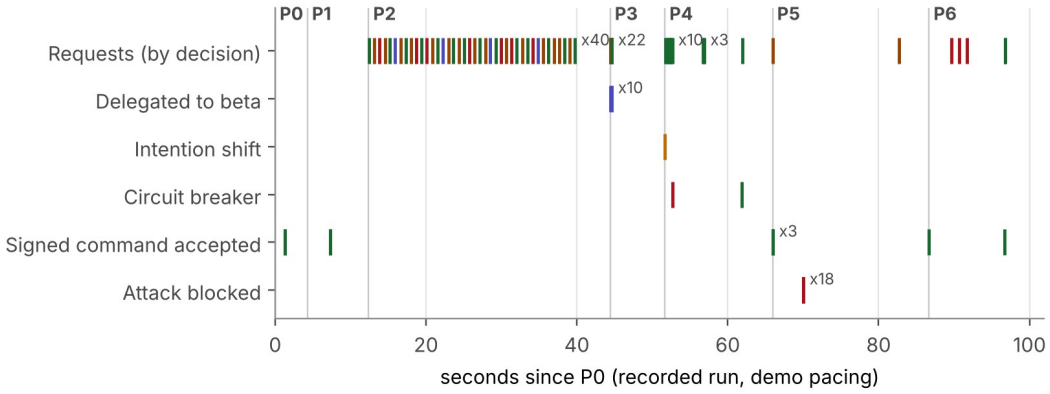


Figure 5.2: Timeline of the recorded run (one tick per event; xN marks a burst)

5.2 P0 Boot and self-attestation

The mesh provisioned three host identities and a release key, wrote the trust store and signed a manifest over 86 source files. Both gateways passed self-attestation in NORMAL mode, loaded the ontology graph (1170 triples, 29 classes) and negotiated a Contract-Net capacity contract with every provider. Table 5.1 shows the contracts the router then used.

Table 5.1: Contract-Net capacity contracts negotiated at boot (USD per 1,000 requests)

Provider	Status	Agreed price	List price
local	AGREED	0.33	0.40
aws-bedrock	AGREED	2.28	2.80
azure-ai	AGREED	2.69	3.30
gcp-vertex	AGREED	1.79	2.20

```
03:41:15.418 [gateway-alpha] ATTEST    self-attestation PASSED: 86 source files match...
03:41:15.430 [gateway-beta] ATTEST    self-attestation PASSED: 86 source files match ...
```

Listing 5: P0 attestation lines from logs/production_cycle.log

5.3 P1 Live knowledge

Gateway-alpha fetched all six public feeds live over TLS (Table 5.2). The live data moved the threat posture to ELEVATED, converted the INR 300 budget at the live rate of 94.91 INR per USD to USD 3.1609 per 1,000 requests, and tightened the R5 prompt limit from 65,536 to 32,768 bytes. Gateway-beta learned the posture and the budget from alpha's signed heartbeat replies and accepted the tighter R5 limit through a field-limited `STATE_SYNC`. Only counts and tags are logged; no headline or description from any feed is stored.

Table 5.2: Live feeds fetched in P1

Feed	Mode	HTTP	Bytes	Items	Relevant	Belief changes
gcp_status	live	200	251,117	8	0	1
azure_status	live	200	577	0	0	1
aws_health	live	200	231,276	2	0	1
cisa_kev	live	200	1,826,755	1766	3	11
fx_usd_inr	live	200	69	1	1	4
aiid_rss	live	200	88,482	100	18	8

```
03:41:21.826 [P1] threat posture -> ELEVATED (ai_incidents: aiid.privacy_per_day=0.57...
03:41:21.826 [P1] fx USD/INR 94.91 -> budget USD 3.1609 per 1k (was 6.0); R5 prompt l...
03:41:23.834 [P1] gateway-beta learned posture=ELEVATED budget=3.1609 from alpha's si...
03:41:23.834 [P1] policy STATE_SYNC alpha -> beta (fields policy_revision, policy_ver...
```

Listing 6: P1 adaptation lines

5.4 P2 Autonomous processing

Forty enterprise prompts went through alpha's four-worker pool: clean questions, e-mail and phone numbers, Aadhaar and PAN, internal URLs, an AWS key, a GitHub token, a Slack token, a database URL, an oversized log and three obfuscated secrets (spaced letters, `[at]`/`[dot]`, Base64). Every prompt got its own BDI deliberation, and every policy decision matched the expectation written in the workload. Table 5.3 summarises the outcome.

Table 5.3: P2 outcome by decision

Decision	Requests	BDI desire and plan	Result
ALLOW	14	D3: `allow_then_dispatch` (best feasible cloud)	sent clean; reply inspected
REDACT	15	D3 under D2: `redact_then_dispatch` or `dispatch_local_only`	masked copy sent; India data only to IN/LOCAL
REVIEW	4	D3: `hold_for_review`	review ticket; nothing sent
BLOCK	7	D1: `block_and_advise_rotation`	evidence sealed; nothing sent

Indian personal data was never routed outside IN or LOCAL providers (residency violations: 0). Figure 5.3 shows the time per request. Blocked and held requests finish in about two milliseconds because nothing is sent; dispatched requests include the simulated provider latency.

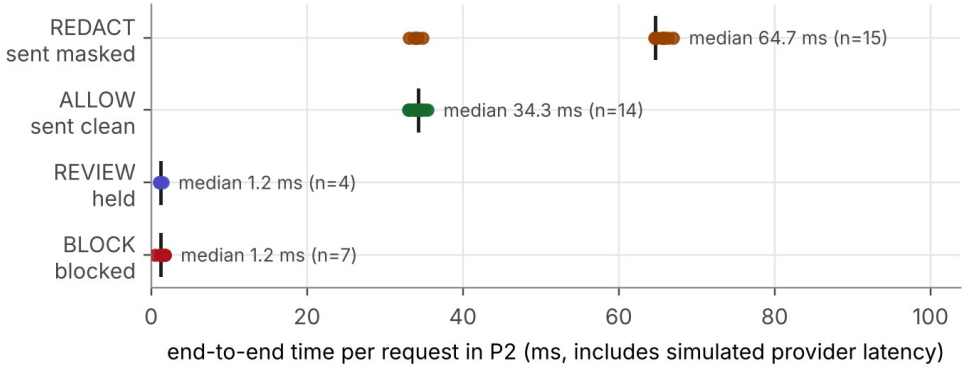


Figure 5.3: End-to-end time per P2 request by decision

```
03:41:55.858 [P2]   03:41:29.942  c01  INTEND     I1 adopted for D3: dispatch_local_o...
03:41:55.858 [P2]   03:41:29.943  c01  STEP       I1 dispatch_local_only(provider=loc...
03:41:55.858 [P2]   03:41:29.943  c02  STEP       I1 dispatch_local_only(provider=loc...
03:41:55.858 [P2]   03:41:29.944  c03  STEP       I1 dispatch_local_only(provider=loc...
03:41:55.858 [P2]   03:41:29.944  c04  STEP       I1 dispatch_local_only(provider=loc...
03:41:55.858 [P2]   03:41:30.005  c05  STEP       I1 dispatch_local_only(provider=loc...
```

Listing 7: BDI trace of the Aadhaar request p2-04 (no prompt text)

5.5 P3 Coordination and load sharing

A burst of 22 prompts arrived at once. When alpha's queue reached the high-water mark of six, alpha announced each shareable task with a signed `PING`, gateway-beta replied with a signed bid, alpha awarded the task as a signed `INSPECT` and beta returned a signed result after its own BDI deliberation. 10 tasks were processed on beta; the 2 BLOCK and REVIEW prompts in the burst stayed on alpha (left the host: 0). Table 5.4 lists the delegated tasks.

Table 5.4: Tasks delegated to gateway-beta in P3

Task	Decision	Bid (ms)	Round trip (ms)	Result
task-alpha-001	**ALLOW**	20	52.1	COMPLETED
task-alpha-002	**ALLOW**	20	85.7	COMPLETED
task-alpha-003	**ALLOW**	20	84.2	COMPLETED
task-alpha-004	**ALLOW**	25.5	86.1	COMPLETED
task-alpha-005	**REDACT**	18.6	58.5	COMPLETED
task-alpha-006	**ALLOW**	18.6	76.7	COMPLETED
task-alpha-007	**ALLOW**	29.1	43.8	COMPLETED
task-alpha-008	**REDACT**	28.2	48.6	COMPLETED
task-alpha-009	**ALLOW**	28	39.4	COMPLETED
task-alpha-010	**ALLOW**	27.6	52.3	COMPLETED

```
03:41:59.864 [gateway-alpha] CFP      task-alpha-001: queue saturated (depth >= 6); ...
03:41:59.865 [gateway-alpha] CFP      task-alpha-002: queue saturated (depth >= 6); ...
03:41:59.867 [gateway-alpha] CFP      task-alpha-003: queue saturated (depth >= 6); ...
03:41:59.876 [gateway-alpha] BID      task-alpha-001: gateway-beta bids accept=True ...
```

Listing 8: one Contract-Net round from the log

5.6 P4 Environmental change: provider outage

While clean traffic was running, a labelled drill switched the simulated GCP Vertex adapter into an HTTP 503 outage just before an intention bound to it reached its `dispatch` step. The belief `provider.gcp-vertex.status = major_outage` reached the running intention through the belief relay, the intention was dropped before transmitting and the agent re-planned (Table 5.5). The health monitor saw consecutive failed probes and opened the circuit breaker. A clearly labelled what-if replay of the Azure status feed then took Azure down too, and beta learned the GCP outage from alpha. After the drill the next healthy probe closed the circuit, Azure was restored from the P1 live snapshot and traffic returned to GCP.

Table 5.5: Intention shifts recorded in P4

Request	Dropped intention	Adopted intention	Reason
p4-01	allow_then_dispatch(provider=gcp-vertex)	dispatch_local_only(provider=local)	requires 'provider gcp-vertex is usable' but believed provider.gcp-vertex.status=major_...

During the outage, successful transmissions to GCP Vertex: 0; circuit opened: true; closed again after recovery: true; beta learned the outage: true.

```
03:42:08.118 [P4]   03:42:07.117  c05  DROP       I1 allow_then_dispatch(provider=gcp...
03:42:08.118 [P4]   03:42:07.119  c06  REPLAN     I1 allow_then_dispatch(provider=gcp...
03:42:08.118 W [gateway-alpha] CIRCUIT   circuit breaker OPEN for gcp-vertex after 3 ...
```

Listing 9: the intention shift and the circuit breaker in the log

5.7 P5 Security

The security console sent three signed commands (a redaction-pattern update, a telemetry dump and a ping), and all were accepted with verified signed replies. A project-codename probe was then redacted by the new pattern. Next the attacker sent 18 hostile frames from untrusted addresses, from a foothold inside the console subnet and with a stolen peer-gateway key. All 18 were blocked, each at the expected checkpoint, and the engine state fingerprint was identical before and after (Table 5.6).

Table 5.6: Attack battery against gateway-alpha

Id	Attack	Source	Check	Reason code
A01	Replay of a captured console command	127.0.0.22	C11	REPLAY_MSG_ID
A02	Forged console signature	127.0.0.22	C8	BAD_SIGNATURE
A03	Payload tampered after signing	127.0.0.22	C8	BAD_SIGNATURE
A04	Unsigned garbage command	127.0.0.66	C2	SCHEMA_VIOLATION
A05	Unknown host identity	127.0.0.66	C5	UNKNOWN_HOST
A06	Captured console frame replayed from outside	127.0.0.66	C7	ORIGIN_NOT_ALLOWED
A07	Oversized frame (declares 1 MiB)	127.0.0.66	C1	FRAME_TOO_LARGE
A08	Protocol downgrade (cm-sig/0)	127.0.0.67	C3	UNSUPPORTED_VERSION
A09	Wrong recipient	127.0.0.67	C4	WRONG_RECIPIENT
A10	Privilege escalation with a peer key	127.0.0.12	C13	COMMAND_NOT_PERMITTED
A11	Compromised peer code	127.0.0.12	C12	ATTESTATION_MISMATCH
A12	Stale command (issued 10 minutes ago)	127.0.0.12	C10	STALE_MESSAGE
B01-B05	Brute-force forged admin command (5 frames)	127.0.0.77	C7	ORIGIN_NOT_ALLOWED
B06	Brute-force forged admin command	127.0.0.77	L0	SOURCE_LOCKED_OUT

Figure 5.4 counts the frames per checkpoint. Every checkpoint from C1 to C13 except C6 and C9 stopped at least one frame in this battery, and the lockout gate L0 stopped the sixth brute-force attempt; C6 and C9 are covered by the Unit 5.1 battery and its tests.

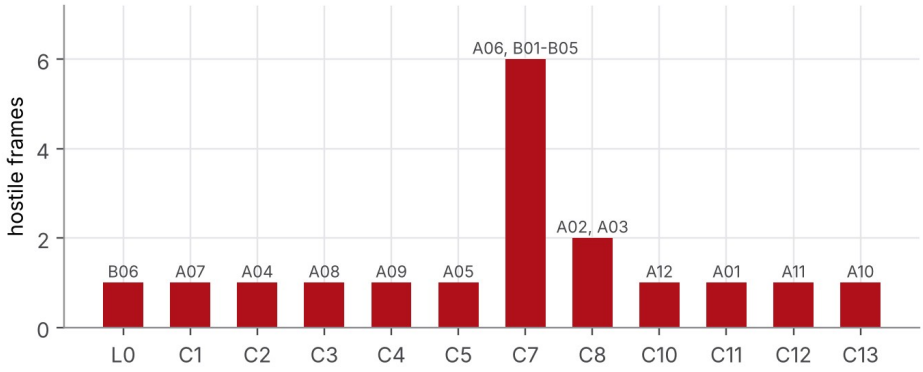


Figure 5.4: Hostile frames blocked per checkpoint in P5

5.8 P6 Lockdown drill and graceful shutdown

The console sent `TRIGGER_EMERGENCY_LOCKDOWN`. Three clean prompts were then BLOCKED by desire D1 with `enforce_lockdown_block`, and provider transmissions during the lockdown were 0. The console released the lockdown with `RELEASE_LOCKDOWN` and the next two prompts completed. Both gateways then drained their queues, stopped their workers and verified their evidence ledgers (Table 5.7).

Table 5.7: Final state of the gateways after the drain

Gateway	Completed	Failed	p50 / p95 (ms)	Ledger records	Ledger check
gateway-alpha	73	0	36.778 / 122.665	125	**VERIFIED**
gateway-beta	10	0	35.78 / 46.376	20	**VERIFIED**

5.9 Results across the cycle

Figure 5.5 shows the decisions per phase. Across the whole run the mesh handled 82 requests (ALLOW 44, REDACT 22, REVIEW 5, BLOCK 11), recorded 1 intention shift, delegated 10 tasks, exchanged 204 signed heartbeats and blocked 18 of 18 attacks. The p50 and p95 request times were 35.5 ms and 69.7 ms.

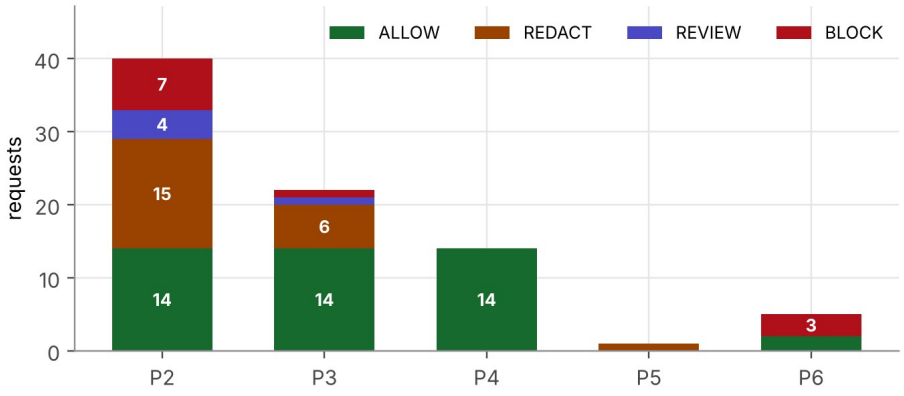


Figure 5.5: Policy decisions per phase in the recorded run

6 TESTING

The full suite is the existing backend tests plus the unit tests copied from Units 4.1, 4.2 and 5.1 and 39 new capstone tests. All 563 pass (`logs/pytest_results.txt`). Table 6.1 gives the counts by unit.

Table 6.1: Test counts by unit

Group	Folder	Tests	What they cover
Units 1-3 engine	`tests/engine/`	112	detector, policy, transport, negotiation, routing, pipeline
v2 API and graph	`tests/test_*.py`	49	REST routes, LangGraph flow, vault
Unit 4.1 BDI	`tests/bdi/`	76	beliefs, desires, plans, intentions, reconsideration
Unit 4.2 Knowledge	`tests/knowledge/`	130	ontology, graph, parsers, fallback, adapter
Unit 5.1 Trust	`tests/trust/`	157	keys, envelope, checkpoints, integrity, breach run
Capstone	`tests/capstone/`	39	host wiring, StateBus, sync, load sharing, ops API, CLI, cycle
Total		563	

Table 6.2 lists the new capstone tests. Hosts in the tests attest a clean copy of the package, because the test process itself may write bytecode caches into the source tree and the Unit 5.1 attestation correctly refuses those.

Table 6.2: Capstone tests

Test file	Tests	Edge cases covered
`test_statebus.py`	6	deep-copy snapshots, version wake-up, bounded ring buffer, faulty subscriber, 6 concurrent writers
`test_coordination.py`	10	policy-sync fields inside the allow-list; India data never delegated; peer and engine beliefs never shared; strict schemas
`test_host.py`	12	all units wired; each decision class; belief sync and policy sync (tighten only); load sharing keeps BLOCK, REVIEW and Aadhaar home; unsigned, weakening, unbound-key and escalation frames blocked; peer refuses BLOCK content; attack battery; lockdown; outage shift; no prompt text in state; `.pyc` forces SAFE MODE
`test_ops_api.py`	5	nonce CSP and no CDN; state JSON; SSE event; token by header and query; v2 routes unchanged
`test_dashboard_cli.py`	4	all sections rendered; ANSI only on a TTY; non-TTY snapshots; unreachable API
`test_cycle.py`	2	workload expectations match the real policy; full P0-P6 cycle offline; no prompt text in logs

7 VIDEO PRESENTATION

The video `video/CipherMesh_Capstone_Lifecycle.mp4` lasts 3:05 at 1920x1080, 30 fps, H.264. Its middle part is a real Playwright recording of the live dashboard during the production cycle whose logs are in `logs/`; title and section cards frame it. Captions are burned in, and

`video/narration_script.md` gives a timed voice-over to read aloud. Table 7.1 is the storyboard and Figure 7.1 shows three key frames.

Table 7.1: Video storyboard

Time	Scene	Content
0:00-0:08	Title	project, student, course, guide, institute
0:08-0:22	Problem	what a prompt can leak; five units become one agent
0:22-0:40	Architecture	the schematic of Figure 3.1
0:40-0:46	Lifecycle	the seven phases
0:46-2:26	Live run P0-P6	dashboard recording with one caption per phase
2:26-2:43	Breach blocked	security card and close-up of the security stream
2:43-2:57	Results	numbers from the run summary
2:57-3:05	Close	tagline and repository

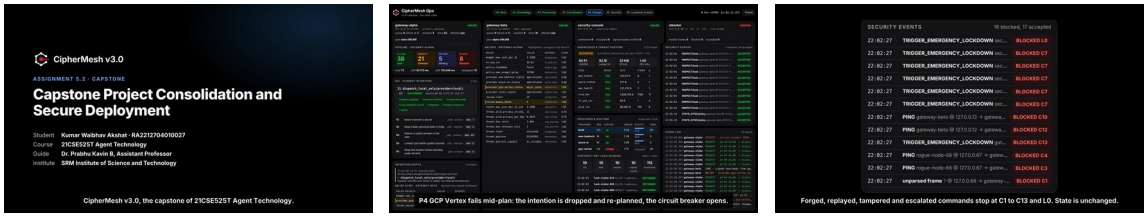


Figure 7.1: Key frames: title card, P4 outage on the live dashboard, security stream close-up

8 SUBMISSION ANALYSIS

8.1 Requirement, evidence and status

Table 8.1 checks each line of the brief against its evidence.

Table 8.1: Requirement, evidence and status

Requirement	Evidence	Status
Unified codebase of Units 1-5	`AgentHost`; unit packages unchanged; wiring test	**PASS**
Clean UI layer with live states	`/ops` (SSE) and CLI; Figures 4.1-4.4; Listing 4	**PASS**
Agent processes inputs independently	P2: 40 prompts, all expectations met	**PASS**
Coordinates with other hosts	P1 belief and policy sync; P3 10 delegated tasks, signed results	**PASS**
Updates its intents	P4: 1 intention shift, circuit breaker	**PASS**
Executes actions safely	P5 18/18 blocked, state unchanged; P6 0 transmissions in lockdown	**PASS**
Source code package and tests	563 tests pass	**PASS**
Repository link	README and `docs/SUBMISSION.md`	**PASS**
3-minute video	3:05, captions, narration script	**PASS**
Architecture schematic	PNG, PDF and HTML source	**PASS**

8.2 Strengths

- **Real integration, not glue scripts.** Every unit runs through its own public API inside one host; a belief from a live feed or a monitor reaches a running plan through the relay.
- **Least privilege between peers.** Coordination needs no new command and no wider role; the field-level `STATE_SYNC` rule of Unit 5.1 is respected, and a stolen peer key cannot lock the mesh down (attack A10).
- **Privacy survives sharing.** Only ALLOW or redacted text crosses hosts, Indian personal data never does, and the receiver re-inspects whatever it is given.
- **Observable.** Both dashboards read one bus; nothing on screen or in the logs contains prompt text.
- **Reproducible.** One command produces every log; the tests run the whole cycle offline.

8.3 Honest limitations

- Cloud providers are simulated adapters with illustrative prices and latencies, as in Units 3 to 4; the local tier calls a real Ollama only when `CIPHERMESH_OLLAMA=1`.
- The GCP outage is a labelled drill on the simulated adapter, and the Azure outage is a labelled what-if replay of a hand-written fixture; on the day of the run the real provider feeds were quiet.
- All hosts run on one machine over loopback. Distinct source addresses make checkpoint C7 meaningful, but a production mesh would add mutual TLS between hosts.
- Attestation is software-only (no TPM), and replay caches and the state bus are in memory, so a restart forgets them.
- Only 1 intention shift happened in the recorded run because only one request was mid-plan when the outage started; the tests and earlier runs show several concurrent shifts.
- The video is silent with burned-in captions; the narration script is provided for a voice-over.

8.4 Lessons learned

- **Integration needs a belief path, not just function calls.** The units fitted together only after host beliefs could reach a running intention on another thread; the `BeliefRelay` is small but it is what makes reconsideration work across the whole host.
- **Security rules shape the protocol.** When the trust unit made `STATE_SYNC` field-level, the first coordination design no longer fitted. Moving reported data into signed replies kept the roles narrow instead of widening them.
- **Provenance prevents echoes.** Without the source of each belief, two gateways would keep sending the same value back and forth and overwrite where it came from. Sharing only observed knowledge solved it.
- **Compiled caches are code.** Self-attestation correctly treats a stray `.pyc` as untrusted, so every run and test uses `-B` or a clean copy of the sources.
- **Show the state, then trust the demo.** The dashboard made bugs visible during development: an empty load-sharing panel exposed that paced submissions never filled the queue.

8.5 Future work

- Mutual TLS and certificate pinning between hosts, with Ed25519 commands on top.
- A persistent, replicated state bus and audit store for more than two gateways.

- Real provider adapters behind the same interface, with budget enforcement on actual invoices.
- Hardware-backed attestation (TPM quotes) feeding checkpoint C12.

9 CONCLUSION

CipherMesh v3.0 joins the reflex policy, the worker pool and sockets, Contract-Net cooperation and utility routing, BDI deliberation with live knowledge, and Ed25519 trust into one autonomous host, and runs two such hosts as a mesh. In the recorded production cycle the agent processed 82 requests on its own, shared 10 tasks with its peer under signed messages, changed its plan when a provider failed mid-execution, and blocked every one of 18 hostile frames without any change to its state. The web and terminal dashboards show all of this live, and the video, logs and tests make it easy to check. The result is a system in which each part from the earlier units is still recognisable, and the whole works as one agent [8] [9].

Table 9.1 maps the main ideas of the course to the place where the recorded run demonstrates them.

Table 9.1: Course concepts and where the capstone demonstrates them

Concept	Demonstrated by	Evidence
Simple-reflex agent	admission reflex on every prompt (R1-R5)	P2 decisions, Table 5.3
Concurrency and daemons	worker pool, daemon health monitor, graceful drain	P6 drain, Table 5.7
Agent communication	typed, signed frames over TCP between four hosts	security events, P5
Negotiation (Contract-Net)	provider contracts; announce, bid, award, result	Tables 5.1 and 5.4
Utility-based choice	constrained router inside every dispatch plan	utility bars, Figure 4.1
BDI and reconsideration	intention dropped and re-planned on a belief change	Table 5.5, Listing 9
Knowledge representation	ontology graph and live feeds turned into beliefs	Table 5.2, Listing 6
Agent security	Ed25519 trust gate, self-attestation, least privilege	Table 5.6, Figure 5.4

Seen through the course, the capstone is a complete agent in the classical sense. It perceives prompts, feeds and peer heartbeats; it keeps beliefs about providers, threats and budgets; it pursues prioritised desires through plans that it revises when the world changes; it cooperates with another agent through a negotiation protocol; and it protects its own percept channel and its own code with cryptography. Each of those ideas was first built and tested alone. The capstone shows that they still hold when they share threads, sockets and state, and that an operator can watch them do so in real time.

REFERENCES

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[2] WHATWG, "Server-sent events," in *HTML Living Standard*. Accessed: Nov. 2, 2026. [Online]. Available: https://html.spec.whatwg.org/multipage/server-sent-events.html

[3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[4] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," Internet Engineering Task Force, RFC 8032, Jan. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8032

[5] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote ATtestation procedureS (RATS) Architecture," Internet Engineering Task Force, RFC 9334, Jan. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9334

[6] R. G. Smith, "The Contract Net Protocol: High-level communication and control in a distributed problem solver," *IEEE Transactions on Computers*, vol. C-29, no. 12, pp. 1104-1113, Dec. 1980.

[7] W3C, "Content Security Policy Level 3," W3C Working Draft. Accessed: Nov. 2, 2026. [Online]. Available: https://www.w3.org/TR/CSP3/

[8] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: Wiley, 2009.

[9] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in *Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS-95)*, San Francisco, CA, USA, 1995, pp. 312-319.

APPENDIX A HOW TO RUN

All commands run from the repository root. The platform needs Python 3.12 or later with `backend/requirements-dev.txt`; the recorder and the video build also need the Python `playwright` package with Chromium and `ffmpeg`.

```
cd backend && pip install -r requirements-dev.txt && cd ..
# production cycle (fast) -> logs/
python -B demo/run_production_cycle.py
# production cycle with the live dashboard at http://127.0.0.1:8000/ops
python -B demo/run_production_cycle.py --serve 8000 --pace 2
# recorded run (cycle + dashboard screenshots + JPEG frames for the video)
PY=python REC_PY=python3 bash demo/record_production_cycle.sh /path/to/frames
# CLI dashboard (live, or one snapshot from a saved state)
cd backend && python -B -m ciphermesh.capstone.dashboard --url http://127.0.0.1:8000
python -B -m ciphermesh.capstone.dashboard --state-file ../logs/ops_state_final.json --once
# tests
python -B -m pytest -v -rs --no-header -p no:logging -o addopts="" > ../logs/
pytest_results.txt
# API + /ops with Docker
cd .. && docker compose up --build
```

Listing 10: commands

Table A.1 lists the exact command that produced each output in the submission.

Table A.1: Commands that produced each output

Output	Command (from the repository root)
`logs/production_*`, `ops_state_final.json`, `cli_dashboard_snapshot.txt`, `security_audit_*.jsonl`, `dashboard_snapshots/`	`PY=backend/.venv/bin/python REC_PY=python3 bash demo/record_production_cycle.sh <frames>`
`logs/pytest_results.txt`	`cd backend && python -B -m pytest -v -rs --no-header -p no:logging -o addopts="" -p no:cacheprovider`
`video/CipherMesh_Capstone_Lifecycle.mp4`, `captions.srt`	`python3 video/src/build_video.py --frames <frames>`
`docs/architecture/CipherMesh_v3_Architecture.png` / `.pdf`	render `CipherMesh_v3_Architecture.html` with Playwright (element `#arch`, scale 3)
`docs/figures/chart_*.png`	`python3 docs/figures/make_charts.py <reportkit>/chart_style.py`
`docs/figures/fig_*.png` diagrams	`node render_diagrams.js docs/figures/diagrams.html docs/figures <ids>`
report `.docx` / `.pdf`	`bash make_report.sh docs/report_src/content.js docs <name>` (course report kit)

APPENDIX B FILE LISTING

Table B.1: Main files of the submission

Path	Content
`backend/ciphermesh/capstone/`	the unified platform (12 modules)
`backend/ciphermesh/{bdi,knowledge,trust}/`	Units 4.1, 4.2 and 5.1 (unchanged)
`backend/app/ops/`	ops routes and the self-contained dashboard page
`backend/tests/`	engine, API, bdi, knowledge, trust and capstone tests
`demo/`	production cycle runner, recorder and wrapper script
`logs/production_cycle.log`	readable log of the recorded run with phase banners
`logs/production_events.jsonl`	every StateBus event of the run, tagged with its phase
`logs/production_summary.json`	per-phase counts, latencies, decisions, shifts, attacks, tasks
`logs/ops_state_final.json`, `cli_dashboard_snapshot.txt`	final dashboard state as JSON and as CLI text
`logs/security_audit_{alpha,beta}.jsonl`	metadata-only trust-gate audit of each gateway
`logs/dashboard_snapshots/`	PNG screenshots at key moments and the capture timeline
`logs/pytest_results.txt`	full test run
`video/`	video, captions, narration script, card and build sources
`docs/architecture/`	schematic PNG, PDF and HTML source
`docs/figures/`, `docs/report_src/`	report figures, chart and diagram sources, report content
`docs/SUBMISSION.md`	how to publish the repository
`frontend/`	the existing Next.js console (unchanged)

APPENDIX C REPOSITORY AND PUBLISHING

The repository is published at https://github.com/kumarwaibhav/CipherMesh. `docs/SUBMISSION.md` lists the commands to initialise the repository with the author's name and e-mail, commit, push to the main branch and tag the release `v3.0.0`; Listing 11 repeats them, one command per line. Private keys are created at run time in a temporary directory and are never committed; `.gitignore` also excludes key files, caches and build output.

```
git init
git config user.name "Kumar Waibhav Akshat"
git config user.email "kwa.waibhav@gmail.com"
git checkout -b main
git add .
git status
git commit -m "CipherMesh v3.0 capstone release (Assignment 5.2)"
git remote add origin https://github.com/kumarwaibhav/CipherMesh.git
git push -u origin main
git tag -a v3.0.0 -m "CipherMesh v3.0.0 capstone release"
git push origin v3.0.0
```

Listing 11: publishing commands (docs/SUBMISSION.md)

Before pushing, `git status` must not list key files, virtual environments, `node_modules/`, `.next/` or Python caches. The video is about 9 MB, well within the normal GitHub file limit. After the push, the README on the repository page shows the architecture image, the live app and API links, the video path and the commands that produced each log.